

EXAMPLE 1 : Medical Diagnosis

Will this patient have Diabetes?

A hospital wants to predict whether a new patient has diabetes based on their medical measurements. They have historical patient records with confirmed diagnoses.

TRAINING DATA (8 Patients)

Patient	Blood Sugar (mg/dL)	BMI	Age	Diagnosis
P1	85	22	25	Healthy
P2	90	24	30	Healthy
P3	180	35	55	Diabetic
P4	160	33	50	Diabetic
P5	95	26	35	Healthy
P6	170	31	48	Diabetic
P7	88	23	28	Healthy
P8	155	30	45	Diabetic

NEW PATIENT (Test Instance T)

Blood Sugar = 130 mg/dL | BMI = 28 | Age = 40

Use $k = 3$, L2 (Euclidean) distance. Predict: Healthy or Diabetic?

STEP 0 — NORMALIZE THE DATA (Critical — scales differ!)

$$\text{Formula: } x_{\text{norm}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}})$$

Feature	Min	Max	Range	T (raw)	T (normalized)
Blood Sugar	85	180	95	130	$(130-85)/95 = 0.47$
BMI	22	35	13	28	$(28-22)/13 = 0.46$
Age	25	55	30	40	$(40-25)/30 = 0.50$

STEP 1 — COMPUTE L2 DISTANCE from T to every patient

L2 formula: $d(T, P) = \sqrt{(BS_T - BS_P)^2 + (BMI_T - BMI_P)^2 + (Age_T - Age_P)^2}$

Patient	Calculation	Distance
P1 (0.00,0.00,0.00)	$\sqrt{(0.47)^2 + (0.46)^2 + (0.50)^2}$	0.827
P2 (0.05,0.15,0.17)	$\sqrt{(0.42)^2 + (0.31)^2 + (0.33)^2}$	0.617
P3 (1.00,1.00,1.00)	$\sqrt{(0.53)^2 + (0.54)^2 + (0.50)^2}$	0.907
P4 (0.79,0.85,0.83)	$\sqrt{(0.32)^2 + (0.39)^2 + (0.33)^2}$	0.603
P5 (0.11,0.31,0.33)	$\sqrt{(0.36)^2 + (0.15)^2 + (0.17)^2}$	0.427
P6 (0.89,0.69,0.77)	$\sqrt{(0.42)^2 + (0.23)^2 + (0.27)^2}$	0.550
P7 (0.03,0.08,0.10)	$\sqrt{(0.44)^2 + (0.38)^2 + (0.40)^2}$	0.706
P8 (0.74,0.62,0.67)	$\sqrt{(0.27)^2 + (0.16)^2 + (0.17)^2}$	0.358

STEP 2 — SORT BY DISTANCE (closest first)

Rank	Patient	Distance	Diagnosis
1st ■	P8	0.358	Diabetic
2nd ■	P5	0.427	Healthy
3rd ■	P6	0.550	Diabetic
4th	P4	0.603	Diabetic
5th	P2	0.617	Healthy
6th	P7	0.706	Healthy
7th	P1	0.827	Healthy
8th	P3	0.907	Diabetic

STEP 3 — SELECT k=3 NEAREST NEIGHBORS → Majority Vote

Neighbor	Distance	Vote
P8 (1st nearest)	0.358	Diabetic
P5 (2nd nearest)	0.427	Healthy
P6 (3rd nearest)	0.550	Diabetic

PREDICTION: DIABETIC

Diabetic votes = 2 | Healthy votes = 1 | Confidence = 2/3 = 67%

BONUS — Distance-Weighted kNN

Neighbor	Distance	Weight (1/d)	Vote
P8	0.358	$1/0.358 = 2.793$	Diabetic
P5	0.427	$1/0.427 = 2.342$	Healthy
P6	0.550	$1/0.550 = 1.818$	Diabetic

Diabetic weighted score = $2.793 + 1.818 = 4.611$ | Healthy weighted score = **2.342** → Still **DIABETIC** (with stronger confidence)

KEY INSIGHT — Effect of k

k=3 → 2 Diabetic, 1 Healthy → **DIABETIC**

k=5 → P8(D), P5(H), P6(D), P4(D), P2(H) → 3 Diabetic, 2 Healthy → **DIABETIC**

k=7 → adds P7(H), P1(H) → 3 Diabetic, 4 Healthy → **HEALTHY (prediction FLIPS!)**

This shows why k selection via cross-validation is critical — never guess!

EXAMPLE 2 : Movie Recommendation

Will this user LIKE the movie?

A streaming platform wants to predict whether a new user will LIKE or DISLIKE a movie. They have ratings from past users on the same movie, along with user profile features.

TRAINING DATA (7 Users)

User	Age	Movies/Week	Action Fan? (1=Yes)	Rating
U1	22	5	1	LIKED
U2	45	1	0	DISLIKED
U3	19	7	1	LIKED
U4	50	2	0	DISLIKED
U5	28	4	1	LIKED
U6	38	3	0	DISLIKED
U7	25	6	1	LIKED

NEW USER (Test Instance T)

Age = 24 | Movies/Week = 5 | Action Fan = 1 (Yes)

Use $k = 3$, L1 (Manhattan) distance. Predict: LIKED or DISLIKED?

STEP 0 — NORMALIZE THE DATA

Feature	Min	Max	Range	T (raw)	T (normalized)
Age	19	50	31	24	$(24-19)/31 = 0.161$
Movies/Week	1	7	6	5	$(5-1)/6 = 0.667$
Action Fan	0	1	1	1	$(1-0)/1 = 1.000$

STEP 1 — COMPUTE L1 DISTANCE from T to every user

L1 formula: $d(T, U) = |Age_T - Age_U| + |Movies_T - Movies_U| + |Action_T - Action_U|$

User	Calculation	Distance
U1 (0.097, 0.667, 1.0)	$ 0.161-0.097 + 0.667-0.667 + 1.0-1.0 = 0.064+0+0$	0.064
U2 (0.839, 0.000, 0.0)	$ 0.161-0.839 + 0.667-0.000 + 1.0-0.0 = 0.678+0.667+1.0$	2.345
U3 (0.000, 1.000, 1.0)	$ 0.161-0.000 + 0.667-1.000 + 1.0-1.0 = 0.161+0.333+0$	0.494
U4 (1.000, 0.167, 0.0)	$ 0.161-1.000 + 0.667-0.167 + 1.0-0.0 = 0.839+0.5+1.0$	2.339
U5 (0.290, 0.500, 1.0)	$ 0.161-0.290 + 0.667-0.500 + 1.0-1.0 = 0.129+0.167+0$	0.296
U6 (0.613, 0.333, 0.0)	$ 0.161-0.613 + 0.667-0.333 + 1.0-0.0 = 0.452+0.334+1.0$	1.786
U7 (0.194, 0.833, 1.0)	$ 0.161-0.194 + 0.667-0.833 + 1.0-1.0 = 0.033+0.166+0$	0.199

STEP 2 — SORT BY DISTANCE

Rank	User	Distance	Rating
1st ■	U1	0.064	LIKED
2nd ■	U7	0.199	LIKED
3rd ■	U5	0.296	LIKED
4th	U3	0.494	LIKED
5th	U6	1.786	DISLIKED
6th	U4	2.339	DISLIKED
7th	U2	2.345	DISLIKED

STEP 3 — SELECT k=3 NEAREST NEIGHBORS → Majority Vote

Neighbor	Distance	Vote
U1 (1st nearest)	0.064	LIKED
U7 (2nd nearest)	0.199	LIKED
U5 (3rd nearest)	0.296	LIKED

PREDICTION: LIKED ■

Liked = 3 | Disliked = 0 | Unanimous! Confidence = 3/3 = 100%

VARYING k — What Changes?

k	Neighbors Selected	Votes	Prediction
k = 1	U1	1 Liked	LIKED
k = 3	U1, U7, U5	3 Liked, 0 Disliked	LIKED
k = 5	U1, U7, U5, U3, U6	4 Liked, 1 Disliked	LIKED
k = 7	All users	4 Liked, 3 Disliked	LIKED

Observation: All values of k give the same prediction here — LIKED. This happens because the data is cleanly separable: action fans who watch many movies form a tight cluster far from non-action fans. When data is clean, k does not matter much. When data is noisy and overlapping (like Example 1), k matters enormously.

SIDE-BY-SIDE COMPARISON

Feature	Example 1: Diabetes	Example 2: Movies
Domain	Healthcare	Entertainment
Features	Blood Sugar, BMI, Age	Age, Movies/Week, Action Fan
Classes	Healthy / Diabetic	Liked / Disliked
Distance used	L2 (Euclidean)	L1 (Manhattan)
k used	3	3
Result	Diabetic (2 vs 1)	Liked (3 vs 0)
Confidence	67% — close vote	100% — unanimous
Normalization	Critical — large scale diff.	Critical — binary vs integer
k sensitivity	k=7 FLIPS the result!	All k values agree

5 UNIVERSAL LESSONS FROM BOTH EXAMPLES

1. Always Normalize. Features on different scales will dominate the distance. Normalize to [0,1] or zero-mean before applying kNN.

2. Distance metric is a modeling choice. L2 amplifies large differences (good for medical data where extreme values matter). L1 treats all differences equally.

3. Confidence matters. kNN gives a natural confidence score: k_{positive} / k . 67% (Example 1) should be treated very differently from 100% (Example 2).

4. Borderline cases are dangerous. In Example 1, changing k from 3 to 7 flips the prediction. Use cross-validation — never guess k.

5. kNN is interpretable. You can always show which past examples drove the prediction. This is valuable in regulated domains like healthcare and finance.